

HARTSEER BACKEND DOCUMENTATION

CAPITULO 1 - DISEÑO DEL BACKEND

INTRODUCCIÓN

El presente documento tiene como finalidad describir el diseño arquitectónico y los principios de implementación adoptados durante el desarrollo del backend de **Hartseer Analytics**, documentando las decisiones técnicas que dieron origen a su arquitectura y las responsabilidades asignadas a cada uno de los componentes que conforman el sistema.

Tras la finalización del modelo de datos, la implementación de la base de datos relacional, el desarrollo de las consultas analíticas, la construcción de los dashboards en Power BI y la posterior implementación del frontend de la aplicación, la siguiente etapa del proyecto consiste en incorporar una API REST como capa de integración entre la interfaz de usuario y la base de datos. Esta nueva arquitectura reemplaza la utilización temporal de archivos JSON empleados durante el desarrollo del frontend, permitiendo que la información sea obtenida de forma dinámica, segura y centralizada.

A diferencia de la arquitectura utilizada durante las primeras etapas del proyecto, donde parte del procesamiento analítico era ejecutado en el navegador debido a la ausencia de un backend, la incorporación de una API permite trasladar toda la lógica de negocio al servidor. De esta manera, el frontend deja de ser responsable del procesamiento de la información y pasa a desempeñar exclusivamente funciones relacionadas con la presentación e interacción con el usuario, mientras que el backend concentra el acceso a los datos, la validación estructural de las solicitudes y respuestas, la ejecución de las reglas de negocio y la construcción de la información consumida por la aplicación.

Durante el proceso de diseño se adoptó una arquitectura basada en capas, donde cada componente posee una única responsabilidad claramente definida. Esta separación de responsabilidades favorece la mantenibilidad del sistema, reduce el acoplamiento entre módulos y facilita la evolución de la aplicación a medida que nuevas funcionalidades sean incorporadas en futuras versiones.

Del mismo modo, el diseño de la API fue concebido siguiendo una filosofía orientada al dominio del negocio y no a la interfaz de usuario. En consecuencia, los contratos definidos por cada endpoint representan entidades y conceptos propios del negocio, evitando que la estructura de las respuestas dependa de componentes visuales específicos del frontend. Esta decisión permite que la misma API pueda ser reutilizada por diferentes consumidores, como la aplicación web, futuras integraciones con Power BI, servicios de exportación, sistemas de monitoreo u otras aplicaciones que requieran acceder a la información analítica del proyecto.

Al igual que en el resto de la documentación técnica de **Hartseer Analytics**, este documento no tiene como objetivo describir la implementación detallada del código fuente, sino documentar las decisiones arquitectónicas adoptadas durante el diseño del backend, explicando el propósito de cada capa, las responsabilidades que asume dentro del sistema y los criterios que justifican su incorporación. De esta manera, la documentación constituye una referencia conceptual de la arquitectura implementada, facilitando tanto su mantenimiento como su evolución en futuras versiones del proyecto.

OBJETIVOS DEL BACKEND

El desarrollo del backend de **Hartseer Analytics** tiene como objetivo incorporar una capa intermedia entre la interfaz de usuario y la base de datos, responsable de centralizar el procesamiento de la información y administrar el acceso a los datos del sistema. Su incorporación responde a la necesidad de reemplazar la arquitectura temporal basada en archivos JSON utilizada durante el desarrollo del frontend por una solución capaz de ofrecer un acceso controlado, centralizado y mantenible a la información analítica del proyecto.

Uno de los principales objetivos de esta arquitectura consiste en desacoplar completamente el frontend de la base de datos relacional. Bajo este enfoque, la interfaz de usuario deja de interactuar directamente con la estructura de MySQL, consumiendo únicamente información procesada y estructurada mediante una API REST. Esta separación permite que ambas capas evolucionen de manera independiente, reduciendo el acoplamiento entre componentes y facilitando futuras modificaciones sin afectar el funcionamiento general de la aplicación.

Del mismo modo, el backend asume la responsabilidad exclusiva de ejecutar toda la lógica de negocio asociada al proyecto. Las validaciones estructurales de las solicitudes, el procesamiento de métricas, la aplicación de reglas analíticas, la agregación de información y la construcción de los resultados dejan de formar parte

del frontend y pasan a gestionarse dentro de la API, distribuidas entre las capas correspondientes. Esta decisión garantiza que toda la información consumida por la aplicación sea generada bajo un conjunto único de reglas, evitando duplicidad de lógica y asegurando la consistencia de los resultados independientemente del cliente que consuma la API.

Otro objetivo fundamental consiste en proteger el acceso a la información almacenada en la base de datos. La implementación de una API elimina la necesidad de exponer directamente la estructura interna del modelo relacional, permitiendo controlar qué información se expone mediante endpoints definidos específicamente para cada necesidad del negocio. De esta manera, el backend actúa como un mecanismo de abstracción que expone únicamente la información necesaria para cada operación, preservando la integridad y seguridad de los datos.

La arquitectura también fue diseñada con un fuerte enfoque en la reutilización y escalabilidad. En lugar de construir respuestas orientadas exclusivamente a los componentes visuales del frontend, los contratos definidos por la API representan entidades y conceptos propios del dominio del negocio. Este enfoque permite que la misma información pueda ser reutilizada por distintos consumidores, como futuras aplicaciones móviles, integraciones con Power BI, sistemas de exportación de reportes, módulos de monitoreo o cualquier otro servicio que requiera acceder a la información analítica del proyecto sin necesidad de modificar la lógica existente.

Finalmente, el backend busca establecer una base arquitectónica preparada para la evolución futura de **Hartseer Analytics**. La adopción de una arquitectura basada en capas, la definición anticipada de contratos estables y la separación estricta de responsabilidades permiten incorporar nuevas funcionalidades, ampliar la cantidad de endpoints o integrar nuevos consumidores sin comprometer la compatibilidad ni la mantenibilidad del sistema. En consecuencia, el backend deja de ser únicamente un mecanismo de comunicación con la base de datos para convertirse en el núcleo responsable de exponer, proteger y administrar toda la lógica de negocio de la aplicación.

ARQUITECTURA GENERAL

La arquitectura del backend de **Hartseer Analytics** fue diseñada siguiendo un modelo basado en capas, donde cada componente asume una responsabilidad específica dentro del flujo de procesamiento de una solicitud. Este enfoque tiene como objetivo separar claramente las distintas responsabilidades del sistema, reduciendo el

acoplamiento entre módulos y favoreciendo la mantenibilidad, escalabilidad y reutilización del código a medida que el proyecto evolucione.

A diferencia de una arquitectura donde la interfaz de usuario interactúa directamente con la base de datos o donde la lógica de negocio se encuentra distribuida entre múltiples componentes, el backend concentra el procesamiento de la lógica de la aplicación dentro de una arquitectura organizada por capas, permitiendo que cada componente asuma una responsabilidad específica dentro del flujo de una solicitud. De esta manera, el sistema mantiene un flujo de ejecución uniforme para todos los endpoints, independientemente del tipo de información solicitada.

La comunicación entre el frontend y la base de datos se realiza exclusivamente a través de una API REST, la cual actúa como punto de entrada único para todas las operaciones del sistema. Las solicitudes dirigidas a los distintos dominios atraviesan una secuencia de componentes especializados, donde la petición es validada, procesada, consultada y transformada según las necesidades de cada operación antes de construir la respuesta correspondiente.

Conceptualmente, el flujo general de procesamiento puede representarse de la siguiente manera:

Frontend

↓

Router

↓

Controller

↓

Service

↓

Repository

↓

MySQL

Dentro de esta arquitectura, cada capa posee una única responsabilidad claramente definida. El **Router** identifica el endpoint solicitado y determina qué componente debe

procesar la operación. Posteriormente, el **Controller** coordina la ejecución de la solicitud delegando toda la lógica de negocio al **Service**, quien concentra el procesamiento analítico, la aplicación de reglas del negocio y la construcción de la información que será consumida por la aplicación.

Para obtener los datos necesarios, el **Service** interactúa exclusivamente con la capa **Repository**, responsable de aislar completamente el acceso a la base de datos. Esta decisión evita que el resto de la aplicación conozca detalles relacionados con consultas SQL, estructuras relacionales o mecanismos de persistencia, permitiendo que cualquier modificación futura sobre la tecnología de almacenamiento tenga un impacto mínimo sobre las capas superiores del sistema.

Una vez recuperada la información desde MySQL, el flujo se ejecuta en sentido inverso. El **Repository** devuelve los datos al **Service**, donde son procesados, agregados y transformados en estructuras orientadas al dominio del negocio. Posteriormente, el resultado retorna al Controller, que lo devuelve a la capa de API para construir la respuesta HTTP correspondiente. FastAPI se encarga de la serialización y validación del contrato definido para el endpoint antes de enviarlo al consumidor.

La adopción de esta arquitectura responde a una decisión de diseño orientada a largo plazo. La separación estricta de responsabilidades permite que cada componente evolucione de manera independiente, facilita la incorporación de nuevas funcionalidades y simplifica las tareas de mantenimiento, depuración y pruebas del sistema. Al mismo tiempo, esta organización establece una base sólida para futuras integraciones, como aplicaciones móviles, herramientas de Business Intelligence, servicios de exportación o sistemas de monitoreo, los cuales podrán consumir la misma API sin necesidad de modificar la lógica de negocio existente.

FLUJO DE PETICIONES

Toda interacción entre el frontend y la base de datos sigue un flujo de procesamiento previamente definido, donde cada solicitud recorre una secuencia de componentes especializados antes de obtener una respuesta. Este recorrido garantiza que todas las operaciones del sistema sean procesadas bajo una misma arquitectura, independientemente del endpoint solicitado, manteniendo un comportamiento uniforme y una clara separación de responsabilidades entre las distintas capas que conforman el backend.

El ciclo comienza cuando el frontend realiza una petición HTTP hacia uno de los endpoints expuestos por la API.

La solicitud es recibida por el **Router**, quien identifica el endpoint solicitado y determina qué controlador será responsable de gestionar la operación. A partir de ese momento, el **Controller** actúa como coordinador del flujo de ejecución, delegando el procesamiento de la lógica de negocio al **Service**, capa donde se concentran todas las reglas analíticas implementadas por la aplicación.

Cuando la lógica requiere acceder a la información almacenada en la base de datos, el **Service** solicita los datos necesarios al **Repository**, responsable exclusivo de establecer la comunicación con MySQL. Esta separación evita que las capas superiores dependan directamente del modelo relacional o de las consultas SQL, favoreciendo una arquitectura desacoplada y fácilmente mantenible.

Una vez recuperada la información, el flujo continúa en sentido inverso. Los datos regresan al **Service**, donde son procesados, transformados y organizados de acuerdo con los contratos definidos para cada endpoint. Posteriormente, el resultado retorna al Controller, que lo devuelve a la capa de API para construir la respuesta HTTP correspondiente. FastAPI se encarga de la serialización y validación del contrato definido para el endpoint antes de enviarlo al frontend.

Este modelo de procesamiento permite que toda la lógica de negocio permanezca centralizada en el backend, mientras que el frontend asume exclusivamente responsabilidades relacionadas con la presentación de la información. Como consecuencia, cualquier consumidor de la API obtiene resultados consistentes, independientemente de la plataforma desde la cual acceda a los datos, manteniendo un comportamiento homogéneo en toda la aplicación.

RESPONSABILIDADES DE CADA CAPA

La arquitectura del backend de **Hartseer Analytics** se encuentra organizada mediante una estructura basada en capas, donde cada componente asume una responsabilidad única y claramente delimitada dentro del flujo de procesamiento de una solicitud. Esta organización responde al principio de **Responsabilidad Única (Single Responsibility Principle)**, buscando que cada módulo del sistema conozca únicamente aquellas tareas que le corresponden, evitando dependencias innecesarias y reduciendo el acoplamiento entre las distintas partes de la aplicación.

En lugar de concentrar toda la lógica del sistema en un único componente, el procesamiento de una petición se distribuye entre diversas capas especializadas que colaboran entre sí siguiendo un flujo predefinido. Cada una de ellas resuelve un problema específico dentro de la arquitectura y delega las responsabilidades restantes a la capa correspondiente. Como consecuencia, la modificación de una determinada funcionalidad suele afectar únicamente al componente responsable de dicha tarea, simplificando considerablemente las labores de mantenimiento, pruebas y evolución del sistema.

Esta separación también favorece la reutilización de la lógica de negocio y permite que la API mantenga un comportamiento homogéneo en todos sus endpoints. Independientemente del recurso solicitado, los distintos recursos mantienen una estructura arquitectónica común, adaptando el procesamiento interno a las necesidades específicas de cada operación, garantizando consistencia en el procesamiento, facilidad para incorporar nuevas funcionalidades y una organización del código alineada con buenas prácticas de desarrollo de software.

A continuación, se describen las responsabilidades asumidas por cada una de las capas que conforman la arquitectura del backend de **Hartseer Analytics**, explicando el propósito de su incorporación y la función que desempeña dentro del flujo general de procesamiento.

4.1 Router

La capa **Router** constituye el punto de entrada lógico de la API y tiene como responsabilidad identificar cada solicitud recibida por el sistema, asociándola con el endpoint correspondiente. Su función consiste en actuar como un mecanismo de enrutamiento que determina qué controlador deberá procesar la operación solicitada, estableciendo el primer nivel de organización funcional dentro de la aplicación.

La incorporación de una capa dedicada exclusivamente al enrutamiento responde a la necesidad de separar la definición de las rutas del resto de la lógica del sistema. Bajo esta arquitectura, el Router no ejecuta validaciones de negocio, no accede a la base de datos ni procesa información analítica. Su responsabilidad principal consiste en definir los endpoints de la API, asociando cada ruta y método HTTP con el componente encargado de gestionar la solicitud y declarando los parámetros y contratos correspondientes.

Esta separación permite que la estructura de la API permanezca organizada y fácilmente escalable a medida que nuevos recursos son incorporados al sistema. Cada conjunto de endpoints puede agruparse según el dominio funcional al que pertenece, manteniendo una organización consistente y facilitando tanto la navegación del código como la incorporación de futuras versiones de la API sin afectar la lógica implementada en otras capas.

Dentro de **Hartseer Analytics**, el Router actúa como un mecanismo de coordinación inicial entre la infraestructura de la API y la lógica de negocio. Una vez identificada la operación solicitada, delega inmediatamente el control al **Controller** correspondiente, manteniendo un desacoplamiento completo entre la estructura de las rutas y el procesamiento interno de la aplicación.

Como consecuencia, cualquier modificación relacionada con la organización de los endpoints, la incorporación de nuevas versiones de la API o la creación de nuevos recursos puede realizarse sin necesidad de alterar la lógica de negocio existente. Esta decisión favorece la mantenibilidad del proyecto, mejora la claridad de la arquitectura y establece una separación precisa entre la definición de la interfaz pública de la API y los componentes responsables de implementar su funcionamiento.

4.2 Controller

La capa **Controller** constituye el punto de coordinación entre la infraestructura de la API y la lógica de negocio implementada por la aplicación. Su responsabilidad consiste en recibir las solicitudes previamente enrutadas por el **Router**, coordinar el flujo general de ejecución y delegar el procesamiento de la operación al **Service** correspondiente, sin participar directamente en la aplicación de reglas de negocio ni en el acceso a la base de datos.

Dentro de la arquitectura de **Hartseer Analytics**, el Controller actúa como un intermediario entre la interfaz pública de la API y las capas encargadas del procesamiento interno. Su función principal consiste en recibir los parámetros de la solicitud, invocar el servicio adecuado y devolver el resultado de la operación a la capa de API.

Esta separación responde al principio de mantener la lógica de coordinación aislada de la lógica de negocio. Bajo este enfoque, el Controller no realiza cálculos analíticos, no ejecuta consultas SQL ni implementa validaciones propias del dominio del negocio. Todas estas responsabilidades son delegadas al **Service**, permitiendo que el Controller permanezca liviano, predecible y fácilmente mantenible.

La adopción de este patrón aporta una importante ventaja desde el punto de vista arquitectónico. Al limitar el Controller exclusivamente a tareas de coordinación, cualquier modificación en las reglas de negocio puede realizarse dentro de la capa de servicios sin afectar la estructura de los endpoints ni la forma en que la API expone sus recursos. Del mismo modo, la lógica implementada por un mismo servicio puede ser reutilizada por distintos controladores o consumidores internos sin generar duplicidad de código.

Como consecuencia, el Controller se convierte en un componente cuya única responsabilidad consiste en gestionar el ciclo de vida de una solicitud dentro de la API, coordinando la comunicación entre las distintas capas sin asumir responsabilidades que pertenecen al dominio del negocio. Esta decisión favorece una arquitectura desacoplada, facilita la realización de pruebas unitarias y mantiene una clara separación entre la definición de la interfaz pública de la API y la implementación de las reglas analíticas que sustentan el funcionamiento de **Hartseer Analytics**.

4.3 Service

La capa **Service** constituye el núcleo funcional del backend de **Hartseer Analytics**, concentrando la totalidad de la lógica de negocio implementada por la aplicación. Su responsabilidad consiste en procesar la información obtenida desde la base de datos, aplicar las reglas analíticas definidas para cada operación y construir las estructuras de datos que posteriormente serán expuestas por la API.

A diferencia del **Controller**, cuya función se limita a coordinar el flujo de una solicitud, el **Service** es el componente responsable de interpretar el dominio del negocio. Todas las decisiones relacionadas con cálculos analíticos, agregaciones, comparaciones temporales, validaciones funcionales, construcción de indicadores y generación de métricas son ejecutadas exclusivamente dentro de esta capa, evitando que dicha lógica se distribuya entre otros componentes de la arquitectura.

Uno de los principios fundamentales adoptados durante el diseño del backend consiste en desacoplar completamente la lógica de negocio de la infraestructura de la aplicación. Como consecuencia, el **Service** no conoce detalles relacionados con la implementación de los endpoints ni con la forma en que la información será presentada por el frontend. Del mismo modo, tampoco establece comunicación directa con la base de datos, delegando el acceso a la información exclusivamente al **Repository**.

Esta separación permite que la lógica analítica permanezca completamente independiente de los mecanismos de persistencia y de la interfaz pública de la API. De esta manera, cualquier modificación sobre las reglas del negocio puede implementarse sin afectar la estructura de los controladores, los contratos de los endpoints o la tecnología utilizada para acceder a la base de datos.

Durante el diseño de **Hartseer Analytics** también se adoptó el principio de **reutilización de la información** dentro de esta capa. La arquitectura prioriza la reutilización de los datos obtenidos y evita consultas redundantes siempre que la operación lo permite. Cuando una operación requiere información adicional, el Service puede solicitar nuevos datos al Repository para completar el procesamiento correspondiente. De esta manera, el procesamiento analítico permanece centralizado en la capa de servicios, manteniendo un equilibrio entre claridad de la lógica y eficiencia en el acceso a los datos.

Bajo esta arquitectura, el **Service** es además responsable de construir respuestas orientadas al dominio del negocio y no a la interfaz de usuario. Los objetos devueltos por la API representan entidades analíticas reutilizables, independientemente de la forma en que posteriormente sean visualizadas por el frontend. Como consecuencia, la misma información puede ser consumida por diferentes clientes, como la aplicación web, futuras integraciones con herramientas de Business Intelligence, sistemas de

exportación de reportes o nuevos módulos incorporados en versiones posteriores del proyecto.

La incorporación de una capa de servicios responde, por lo tanto, a una decisión de diseño orientada a centralizar el conocimiento del negocio en un único punto de la arquitectura. Esta organización favorece la mantenibilidad del sistema, simplifica la incorporación de nuevas funcionalidades y garantiza que todas las reglas analíticas sean ejecutadas de forma consistente, independientemente del consumidor que acceda a la API.

4.4 Repository

La capa **Repository** constituye la abstracción utilizada por los servicios para acceder a la información persistida. Su responsabilidad consiste en encapsular las operaciones de acceso a datos y ejecutar las consultas necesarias para recuperar la información requerida por la lógica de negocio, manteniendo los detalles de persistencia separados de las capas superiores

Dentro de la arquitectura de **Hartseer Analytics**, el Repository actúa como una capa de abstracción entre el **Service** y MySQL. Bajo este enfoque, los servicios no conocen la estructura física de la base de datos, las consultas SQL implementadas ni los mecanismos utilizados para obtener la información. Su única responsabilidad consiste en solicitar los datos necesarios al Repository, delegando completamente el acceso a la persistencia.

La separación reduce el impacto que las modificaciones relacionadas con la estructura de la base de datos o la optimización de consultas pueden tener sobre las capas superiores de la aplicación.

A diferencia del **Service**, cuya responsabilidad consiste en interpretar la información desde una perspectiva analítica, el Repository no aplica reglas de negocio, no realiza cálculos ni construye estructuras orientadas al dominio funcional de la aplicación. Su propósito es recuperar los datos solicitados de la forma más eficiente posible y devolverlos al Service para que este continúe con el procesamiento correspondiente.

Durante el diseño de la arquitectura también se adoptó el criterio de mantener las consultas SQL centralizadas dentro de esta capa. Esta decisión evita la duplicación de código, facilita la optimización de consultas y simplifica las tareas de mantenimiento cuando resulta necesario modificar la forma en que la información es obtenida desde MySQL. Del mismo modo, la concentración del acceso a datos en un único

componente favorece la realización de pruebas unitarias y permite validar de forma independiente tanto la lógica de negocio como la persistencia de la información.

La incorporación de una capa Repository responde, por lo tanto, a una decisión orientada a preservar la independencia entre el dominio del negocio y la infraestructura de almacenamiento. Esta organización fortalece la mantenibilidad del sistema, mejora la reutilización de los mecanismos de acceso a datos y establece una arquitectura preparada para evolucionar sin que las modificaciones realizadas sobre la base de datos afecten directamente al resto de la aplicación.

4.5 Database (MySQL)

La base de datos constituye la capa de persistencia del backend de **Hartseer Analytics**, siendo responsable del almacenamiento permanente de toda la información utilizada por la aplicación. Su función consiste en conservar la información del negocio y proporcionar un mecanismo confiable para su almacenamiento y consulta.

Dentro del diseño del sistema, MySQL representa el nivel más bajo de la arquitectura y permanece completamente aislado del resto de los componentes mediante la capa **Repository**. Esta decisión evita que controladores, servicios o cualquier otro módulo de la aplicación establezcan comunicación directa con la base de datos, garantizando que toda interacción con la información persistida se realice de forma centralizada y controlada.

La información almacenada en MySQL corresponde al modelo relacional previamente diseñado durante las etapas iniciales del proyecto, el cual constituye la fuente única de datos sobre la que se apoyan todos los procesos analíticos implementados por la aplicación. Como consecuencia, la base de datos actúa como el repositorio oficial de la información, mientras que la API se encarga de interpretar dichos datos y transformarlos en estructuras orientadas al dominio del negocio.

Es importante destacar que la base de datos no posee conocimiento alguno sobre las reglas analíticas implementadas por la aplicación. Su responsabilidad se limita exclusivamente al almacenamiento y recuperación eficiente de la información, mientras que toda decisión relacionada con cálculos, agregaciones, comparaciones temporales o construcción de indicadores es ejecutada por la capa **Service**. Esta separación permite mantener claramente diferenciadas las responsabilidades de persistencia y procesamiento, evitando trasladar lógica de negocio hacia el modelo de datos.

La incorporación de MySQL como mecanismo de persistencia responde a los requerimientos funcionales y analíticos del proyecto, proporcionando un sistema relacional robusto capaz de gestionar eficientemente grandes volúmenes de información estructurada y facilitar la ejecución de consultas complejas. Sin embargo, desde una perspectiva arquitectónica, la aplicación no depende directamente del motor de base de datos utilizado, sino de la abstracción proporcionada por la capa **Repository**. Esta decisión reduce el acoplamiento tecnológico y permite que futuras modificaciones sobre la infraestructura de almacenamiento puedan abordarse con un impacto mínimo sobre el resto del sistema.

En conjunto, la base de datos constituye el fundamento sobre el cual se construye toda la arquitectura analítica de **Hartseer Analytics**, proporcionando una fuente consistente y centralizada de información, mientras que las capas superiores de la aplicación se encargan de transformar dichos datos en conocimiento útil para los distintos consumidores de la API.

FILOSOFIA DE DISEÑO

El diseño del backend de **Hartseer Analytics** fue concebido bajo una filosofía orientada a la construcción de una arquitectura mantenible, desacoplada y preparada para evolucionar junto con el proyecto. Más allá de la implementación tecnológica seleccionada, todas las decisiones arquitectónicas fueron tomadas siguiendo un conjunto de principios cuyo objetivo consiste en garantizar la consistencia del sistema, facilitar su mantenimiento y permitir la incorporación de nuevas funcionalidades sin comprometer la estabilidad de la aplicación.

Uno de los principios fundamentales adoptados durante el diseño consiste en considerar al backend como el núcleo responsable de la lógica del negocio. En consecuencia, toda regla analítica, validación, procesamiento de información y construcción de indicadores es ejecutada exclusivamente dentro de la API, evitando trasladar responsabilidades al frontend. Bajo este enfoque, la interfaz de usuario deja de interpretar los datos obtenidos desde la base de datos y pasa a consumir información previamente procesada, concentrando su responsabilidad únicamente en la representación visual y la interacción con el usuario.

Del mismo modo, la arquitectura fue diseñada siguiendo una estricta separación de responsabilidades entre sus distintas capas. Cada componente posee una única función claramente definida dentro del sistema y delega el resto de las operaciones al componente especializado correspondiente. Esta organización reduce el acoplamiento

entre módulos, facilita la reutilización del código y simplifica la incorporación de nuevas funcionalidades sin afectar el comportamiento del resto de la aplicación.

Otro principio adoptado durante el proceso de diseño consiste en orientar la API al dominio del negocio y no a la interfaz de usuario. Los contratos definidos por cada endpoint representan entidades, indicadores y conceptos analíticos propios del negocio, evitando construir respuestas condicionadas por la forma en que posteriormente serán visualizadas por el frontend. Como consecuencia, una misma respuesta puede ser reutilizada por diferentes consumidores, como aplicaciones web, futuras aplicaciones móviles, herramientas de Business Intelligence, sistemas de exportación o cualquier otro servicio que requiera acceder a la información analítica del proyecto.

Con el mismo criterio, todas las respuestas de la API fueron diseñadas bajo una **estructura general** estable y homogénea. Cada endpoint mantiene una estructura consistente para la representación de datos, metadatos, indicadores y errores, permitiendo que cualquier consumidor pueda interpretar las respuestas siguiendo un único criterio de integración. Esta decisión favorece la mantenibilidad del sistema y reduce el impacto que futuras ampliaciones puedan generar sobre los clientes que consumen la API.

La arquitectura prioriza un acceso eficiente a los datos y evita consultas redundantes cuando la operación lo permite. Cuando el procesamiento requiere información adicional, la capa Service puede solicitarla al Repository, manteniendo en todo momento la separación entre lógica de negocio y acceso a datos.

Otro aspecto considerado desde las primeras etapas del diseño fue la preparación para la evolución futura del proyecto. La arquitectura fue concebida para incorporar nuevas funcionalidades mediante la extensión de la API, priorizando la compatibilidad hacia atrás y evitando modificaciones incompatibles sobre los contratos existentes. Este enfoque permite que futuras versiones de **Hartseer Analytics** puedan incorporar nuevos endpoints, ampliar la información expuesta o integrar nuevos consumidores sin comprometer el funcionamiento de las implementaciones ya existentes.

Finalmente, todas las decisiones arquitectónicas adoptadas durante el desarrollo del backend responden a una misma filosofía de diseño: construir una API que represente el conocimiento del negocio y no simplemente un mecanismo de acceso a la base de datos. Bajo esta perspectiva, la base de datos constituye la fuente de información del sistema, mientras que la API se convierte en el componente responsable de interpretar dichos datos, aplicar las reglas analíticas correspondientes y exponer una representación consistente, reutilizable y preparada para acompañar el crecimiento futuro de **Hartseer Analytics**.

DISEÑO DE CONTRATOS

Uno de los principios fundamentales adoptados durante el diseño de la API consiste en mantener contratos de comunicación homogéneos, estables y orientados al dominio del negocio. Cada endpoint fue concebido para exponer información analítica mediante una estructura consistente, permitiendo que cualquier consumidor de la API pueda interpretar las respuestas siguiendo un único criterio de integración, independientemente del recurso solicitado.

En lugar de construir respuestas específicas para los componentes visuales del frontend, los contratos fueron diseñados para representar entidades, métricas e indicadores propios del negocio. Esta decisión permite desacoplar completamente la lógica de presentación de la estructura de la información, favoreciendo la reutilización de los datos por diferentes consumidores sin necesidad de modificar la API cuando la interfaz evolucione o incorpore nuevas visualizaciones.

Con el propósito de mantener una experiencia de integración uniforme, todos los endpoints comparten una estructura general de respuesta basada en un conjunto común de datos y metadatos. Mientras el bloque principal contiene la información analítica correspondiente a cada recurso, los metadatos proporcionan contexto adicional sobre la respuesta procesada, incluyendo información común como la versión de la API y la marca temporal de generación de la respuesta.

La representación de los indicadores se estructura mediante schemas específicos para cada dominio analítico. Cada contrato define las métricas y colecciones necesarias para representar la información de su respectivo dashboard, manteniendo una estructura consistente dentro de cada endpoint. Esta organización permite validar las respuestas y mantener contratos claros sin forzar una estructura idéntica entre dominios con necesidades analíticas diferentes.

La misma filosofía fue aplicada al diseño de las colecciones utilizadas por los distintos endpoints. En lugar de construir estructuras independientes para cada gráfico o componente visual, las respuestas agrupan la información según entidades del dominio del negocio, como productos, clientes o canales comerciales.

Posteriormente, cada consumidor decide cómo representar dicha información según sus necesidades específicas. Como consecuencia, una única colección puede alimentar múltiples visualizaciones, evitando la duplicación de datos y reduciendo la complejidad de la API.

Otro aspecto considerado durante el diseño fue la estandarización del tratamiento de errores. Las respuestas de error generadas por la propia aplicación mantienen un

formato común, incorporando un código, un mensaje y, cuando corresponde, información adicional sobre el error. Esta decisión permite que los consumidores interpreten los errores de forma consistente, reaccionando principalmente a códigos de error y estados HTTP estandarizados en lugar de depender de mensajes descriptivos, favoreciendo una integración más robusta y preparada para futuras evoluciones.

Finalmente, los contratos fueron concebidos con un fuerte enfoque en la compatibilidad hacia atrás. La incorporación de nuevas funcionalidades dentro de **Hartseer Analytics** se realizará mediante la extensión de los contratos existentes o la incorporación de nuevos endpoints, evitando modificaciones incompatibles que obliguen a actualizar los consumidores ya implementados. Esta estrategia permite que la API evolucione de forma progresiva, preservando la estabilidad de las integraciones existentes y garantizando una base sólida para futuras versiones del proyecto.

DISEÑO DE LOS ENDPOINTS

Los endpoints se encuentran agrupados bajo la versión `/api/v1`, estableciendo una primera versión explícita del contrato público de la API. Actualmente, la aplicación expone los recursos `health`, `executive`, `products`, `customers` y `marketing`.

La API de **Hartseer Analytics** fue diseñada siguiendo una organización basada en dominios funcionales del negocio. En lugar de estructurar los recursos según tablas de la base de datos o componentes específicos del frontend, cada endpoint representa un área analítica independiente dentro de la aplicación, agrupando la información necesaria para responder a una necesidad concreta del proceso de toma de decisiones.

Esta organización permite mantener una clara separación entre los distintos ámbitos analíticos del negocio, facilitando tanto la evolución independiente de cada recurso como la incorporación de nuevas funcionalidades sin afectar la estructura general de la API. Al mismo tiempo, cada endpoint mantiene los principios arquitectónicos definidos durante el diseño, compartiendo contratos homogéneos, criterios de validación consistentes y una estructura común de respuestas.

El endpoint **Executive** fue concebido como el punto de acceso principal para el análisis global del negocio. Su propósito consiste en proporcionar una visión consolidada del rendimiento general de la organización mediante indicadores estratégicos, tendencias temporales y métricas agregadas que permitan evaluar la evolución del negocio desde una perspectiva ejecutiva.

Por su parte, el endpoint **Products** fue diseñado para analizar el desempeño comercial del catálogo de productos desde distintas dimensiones de análisis. El endpoint permite seleccionar la dimensión de agrupamiento mediante el parámetro `group_by`, obteniendo información agregada por categorías, marcas o productos individuales.

El endpoint **Customers** se orienta al análisis del comportamiento de la cartera de clientes durante el período seleccionado. Su diseño incorpora indicadores relacionados con adquisición, recurrencia, volumen de operaciones y rentabilidad, permitiendo evaluar la evolución de la base de clientes desde una perspectiva comercial sin depender de la representación visual utilizada posteriormente por el frontend.

Finalmente, el endpoint **Marketing** concentra toda la información vinculada al desempeño de los distintos canales comerciales. Su propósito consiste en facilitar la comparación del rendimiento entre canales mediante indicadores asociados a ingresos, rentabilidad, inversión publicitaria y retorno sobre la inversión, proporcionando una visión unificada del impacto generado por cada estrategia de comercialización.

La definición de estos cuatro dominios funcionales responde a la estructura analítica previamente implementada en el frontend y en los dashboards desarrollados con Power BI, permitiendo mantener coherencia entre las distintas etapas del proyecto. Sin embargo, la API fue diseñada de manera completamente independiente de dichas interfaces, representando entidades y conceptos propios del negocio en lugar de componentes visuales específicos.

Como consecuencia, cada endpoint constituye un servicio analítico reutilizable capaz de ser consumido por múltiples aplicaciones sin necesidad de modificaciones adicionales. Esta organización proporciona una base sólida para la incorporación de nuevos recursos en futuras versiones de **Hartseer Analytics**, preservando la consistencia arquitectónica y la compatibilidad con las integraciones existentes.

CAPITULO 2 - IMPLEMENTACIÓN DE LA API

ESTRUCTURA DEL PROYECTO

La implementación del backend se organiza dentro del directorio backend/, utilizando una estructura modular que separa la configuración, el acceso a datos, el procesamiento de cada dominio, las rutas de la API y los contratos de respuesta.

backend/

|

├─ app/

|

├─ controllers/

|

├─ core/

|

├─ database/

|

├─ repositories/

|

├─ routes/

|

├─ schemas/

|

└─ services/

|

├─ .env

|

└─ requirements.txt

Dentro de app/, cada directorio agrupa componentes con una función específica dentro de la implementación:

- **controllers/** contiene los componentes que reciben las operaciones desde las rutas y coordinan la ejecución del dominio correspondiente.
- **core/** concentra la configuración general de la aplicación y los componentes comunes utilizados por el backend.
- **database/** contiene la infraestructura relacionada con la conexión a MySQL y la obtención de conexiones desde el pool.
- **repositories/** contiene las operaciones de acceso a datos y las consultas utilizadas para recuperar la información analítica desde MySQL.

- **routes/** define los endpoints expuestos por la API y sus parámetros de entrada.
- **schemas/** contiene los modelos Pydantic utilizados para definir y validar los contratos de las respuestas.
- **services/** contiene la lógica de procesamiento específica de cada dominio analítico.

Los dominios principales de la API se mantienen separados dentro de estas capas, utilizando componentes específicos para **Executive, Products, Customers y Marketing**, mientras que health proporciona el endpoint destinado a comprobar el estado de la aplicación.

Esta estructura permite que la implementación mantenga una separación física entre los distintos tipos de componentes sin mezclar consultas SQL, lógica de negocio, definición de endpoints y contratos de respuesta dentro de los mismos archivos.

El archivo requirements.txt contiene las dependencias necesarias para ejecutar el backend, mientras que las variables de configuración sensibles y específicas del entorno se proporcionan mediante .env y no forman parte de la lógica de la aplicación.

CONFIGURACIÓN Y VARIABLES DE ENTORNO

La configuración del backend se centraliza en app/core/config.py, donde se definen los parámetros generales de la aplicación y los valores necesarios para establecer la conexión con MySQL.

La clase Settings concentra principalmente tres grupos de configuración:

- **Configuración de la aplicación**
 - API_TITLE
 - API_DESCRIPTION
 - API_VERSION
- **Configuración del entorno**
 - ENVIRONMENT
 - DEBUG
- **Configuración de la base de datos**
 - DB_HOST
 - DB_PORT
 - DB_NAME
 - DB_USER
 - DB_PASSWORD

Los valores específicos del entorno se obtienen mediante variables de entorno utilizando python-dotenv. El archivo .env se carga al inicializar el módulo de configuración, evitando que las credenciales y parámetros de infraestructura deban incorporarse directamente al código fuente.

La configuración queda disponible mediante una única instancia que posteriormente es utilizada por los distintos componentes del backend que requieren acceso a estos valores. Por ejemplo, la configuración de base de datos es consumida por la capa responsable de establecer el pool de conexiones con MySQL.

Esta organización permite mantener separada la configuración del entorno de la lógica de aplicación y facilita utilizar la misma implementación en diferentes entornos modificando únicamente sus variables de configuración.

CONEXIÓN Y GESTIÓN DE BASE DE DATOS

La conexión con MySQL se implementa mediante el conector oficial utilizado por el backend y un sistema de **pool de conexiones**. La infraestructura de base de datos se concentra en el módulo `app/database/`, compuesto por los componentes responsables de inicializar el pool y proporcionar conexiones disponibles al resto de la aplicación.

El pool se configura con una capacidad de **10 conexiones**, utilizando los parámetros definidos en la configuración de la aplicación. Las credenciales y demás datos específicos de conexión permanecen externalizados mediante variables de entorno.

El acceso a las conexiones se centraliza mediante una función de sesión que obtiene una conexión disponible desde el pool. De esta forma, los componentes que requieren acceso a MySQL utilizan la infraestructura común en lugar de crear conexiones independientes.

La capa de conexión no contiene lógica analítica ni consultas específicas del negocio. Su responsabilidad se limita a proporcionar y gestionar el acceso a la base de datos, mientras que las operaciones de consulta se encuentran implementadas en los repositorios correspondientes.

Esta separación permite mantener aislada la infraestructura de persistencia y evita que los componentes encargados del procesamiento de cada dominio deban gestionar directamente la creación de conexiones.

IMPLEMENTACIÓN DE RUTAS Y FLUJO INTERNO

Las rutas de la API se encuentran agrupadas en `app/routes/` y definen los endpoints disponibles para cada dominio. La implementación actual expone los recursos `health`, `executive`, `products`, `customers` y `marketing`, agrupados bajo la versión `/api/v1`.

Cada ruta declara los parámetros necesarios para ejecutar la operación correspondiente y, cuando aplica, el schema utilizado para validar la respuesta. Las rutas no contienen la lógica de procesamiento de los datos, sino que delegan la operación al controller correspondiente.

El flujo interno de una solicitud sigue la siguiente secuencia:

Request

↓

Route

↓

Controller

↓

Service

↓

Repository

↓

MySQL

↓

Repository

↓

Service

↓

Controller

↓

Response

La **Route** recibe la solicitud y sus parámetros, determina el recurso solicitado y delega la operación.

El **Controller** recibe los parámetros proporcionados por la ruta e invoca el componente de servicio correspondiente al dominio.

El **Service** ejecuta el procesamiento necesario para construir el resultado solicitado. Cuando requiere información persistida, solicita los datos correspondientes al Repository.

El **Repository** ejecuta las consultas necesarias contra MySQL y devuelve los resultados al Service.

Una vez completado el procesamiento, el resultado retorna a través de las capas hasta la ruta, donde FastAPI aplica el contrato de respuesta definido para el endpoint y genera la respuesta HTTP que será enviada al consumidor.

Esta implementación mantiene las rutas enfocadas en la exposición de la API y evita incorporar consultas SQL o lógica analítica directamente en los endpoints.

SCHEMAS Y CONTRATOS

Los contratos de respuesta de la API se implementan mediante **Pydantic**, utilizando schemas específicos para cada dominio. Estos modelos definen la estructura que debe presentar la información retornada por cada endpoint y permiten que FastAPI valide las respuestas antes de entregarlas al consumidor.

Los schemas se encuentran organizados en app/schemas/:

schemas/

└─ common.py

└─ executive.py

└─ products.py

└─ customers.py

└─ marketing.py

└─ health.py

common.py contiene las estructuras compartidas entre los distintos dominios, mientras que los archivos restantes definen los contratos específicos de cada endpoint.

La respuesta general de la API mantiene una estructura común compuesta por tres elementos principales:

Success

Data

Meta

success indica si la operación fue completada correctamente. data contiene la información específica del recurso solicitado y meta proporciona información común asociada a la respuesta, como la versión de la API y su marca temporal.

Los contenidos de data varían según el dominio. Por ejemplo, Products incorpora información agrupada según la dimensión solicitada, mientras que Marketing incluye sus métricas principales y la información correspondiente a sus tendencias. De esta manera, los contratos mantienen una estructura general consistente sin forzar que todos los dominios utilicen exactamente las mismas estructuras internas.

Los schemas también permiten mantener desacoplada la representación externa de la API respecto de las estructuras internas utilizadas durante el procesamiento. Los Services y Repositories pueden trabajar con sus propios modelos de datos, mientras que el contrato expuesto al consumidor queda definido explícitamente en la capa de schemas.

En las rutas que utilizan response_model, FastAPI emplea estos contratos para validar y serializar la respuesta antes de enviarla al cliente. Esto proporciona una interfaz predecible para el frontend y permite detectar incompatibilidades entre los datos generados por el backend y la estructura esperada por la API.

RESPUESTAS DE LA API

Las respuestas de la API utilizan una estructura estandarizada para mantener un formato consistente entre los distintos endpoints. La estructura diferencia entre operaciones exitosas y respuestas generadas ante errores de la aplicación.

Respuestas exitosas

Las operaciones completadas correctamente utilizan una estructura compuesta por:

Success

Data

Meta

success indica que la operación fue completada correctamente y se establece en true.

data contiene el resultado específico de la operación. Su estructura depende del endpoint solicitado y corresponde al schema definido para el dominio.

meta contiene información común de la respuesta, incluyendo la versión de la API y la marca temporal correspondiente a su generación. Cuando una operación requiere información adicional, este bloque puede incorporar metadatos específicos.

Respuestas de error

Las respuestas de error generadas por la propia aplicación mantienen una estructura diferenciada:

Success

Error

Meta

En este caso, success se establece en false, mientras que error contiene la información asociada al problema detectado.

El bloque de error contempla:

- **code** — identificador del tipo de error.
- **message** — descripción del error.
- **details** — información adicional cuando resulta necesaria.

El bloque meta mantiene la información común de la respuesta, permitiendo conservar la versión de la API y la marca temporal incluso cuando la operación no puede completarse correctamente.

Estandarización

La construcción de estas respuestas se centraliza en `app/core/responses.py`, evitando que cada endpoint tenga que implementar individualmente la misma estructura de respuesta.

Esta estandarización permite que el frontend pueda interpretar de forma consistente el resultado de las distintas operaciones y diferencia claramente entre los datos obtenidos correctamente y los errores generados durante el procesamiento.

IMPLEMENTACIÓN POR DOMINIO

La API organiza su implementación alrededor de los dominios funcionales utilizados por el frontend. Cada dominio dispone de los componentes necesarios dentro de las capas de rutas, controllers, services, repositories y schemas, manteniendo el mismo flujo general de procesamiento definido para la aplicación.

Los dominios implementados son **Health, Executive, Products, Customers y Marketing**.

Health

El endpoint `/api/v1/health` proporciona un mecanismo para comprobar el estado operativo de la API.

Su implementación se encuentra separada del procesamiento analítico de los dashboards y permite verificar que la aplicación se encuentra disponible y puede responder correctamente a una solicitud de estado.

La respuesta utiliza un schema específico de Health y mantiene la estructura general de respuestas definida por la API.

Executive

El endpoint `/api/v1/executive` proporciona los datos utilizados por el Executive Dashboard.

La solicitud recibe un período mediante `start_date` y `end_date`. Estos parámetros determinan el intervalo temporal utilizado durante el procesamiento de los datos.

El Service de Executive procesa la información obtenida desde el Repository y construye las métricas requeridas por el dashboard, mientras que el schema correspondiente define la estructura de la respuesta expuesta al frontend.

Products

El endpoint `/api/v1/products` proporciona los datos utilizados por el Products Dashboard.

Además del período mediante `start_date` y `end_date`, la solicitud recibe el parámetro `group_by`, que determina la dimensión utilizada para agrupar los resultados.

La implementación permite trabajar con las dimensiones disponibles para el análisis de productos, generando los resultados financieros y operativos correspondientes a la agrupación solicitada.

El resultado se valida mediante `ProductsResponseSchema` antes de ser enviado al frontend.

Customers

El endpoint `/api/v1/customers` proporciona la información utilizada por el Customers Dashboard.

La solicitud recibe `start_date` y `end_date` para establecer el período analizado. El procesamiento posterior obtiene y transforma la información necesaria para generar las métricas relacionadas con clientes y recurrencia.

La respuesta utiliza `CustomersResponseSchema`, que define explícitamente la estructura de los datos entregados al frontend.

Marketing

El endpoint `/api/v1/marketing` proporciona los datos utilizados por el Marketing Dashboard.

La solicitud recibe `start_date`, `end_date` y `channel`. El parámetro `channel` permite solicitar información correspondiente a un canal específico o utilizar `ALL` para obtener información agregada sobre todos los canales disponibles.

La respuesta incluye las métricas principales de marketing y la información de tendencias utilizada por los gráficos del dashboard. Esta última permite representar la evolución de los resultados por período y canal.

La estructura de la respuesta se encuentra definida mediante `MarketingResponseSchema`, incluyendo los modelos correspondientes a las métricas principales y a los elementos de tendencias.

Resumen de implementación

Los cinco dominios mantienen el mismo patrón general de integración:

Endpoint

↓

Controller

↓

Service

↓

Repository

↓

MySQL

Las diferencias entre dominios se encuentran principalmente en los parámetros recibidos, las consultas ejecutadas, las transformaciones realizadas y la estructura específica de los datos retornados. Esto permite mantener una implementación uniforme a nivel arquitectónico sin forzar que todos los dashboards utilicen el mismo modelo de datos.

CAPITULO 3 - CONFLICTOS Y ERRORES DETECTADOS

AUDITORÍA INICIAL

Una vez completada la integración entre el frontend, la API y MySQL, se realizó una auditoría general del proyecto con el objetivo de comprobar que la implementación final coincidiera con la arquitectura definida y que no permanecieran componentes pertenecientes a etapas anteriores del desarrollo.

La revisión contempló tanto el backend como el frontend, considerando especialmente:

- estructura de directorios y responsabilidades de cada componente;
- correspondencia entre rutas, controllers, services, repositories y schemas;
- consistencia entre las respuestas reales de la API y sus contratos definidos;
- funcionamiento de los cuatro dashboards mediante la API;
- existencia de código heredado de la arquitectura basada en archivos JSON;
- referencias a componentes que ya no formaban parte del flujo actual;
- errores funcionales detectados durante las pruebas;
- puntos susceptibles de optimización dentro del frontend.

La auditoría se realizó de forma incremental. Cada modificación fue seguida por una ejecución local del backend y una comprobación funcional del frontend, evitando realizar cambios simultáneos sobre múltiples componentes sin verificar previamente su impacto.

Este proceso permitió distinguir entre componentes actualmente utilizados, elementos históricos que podían conservarse como referencia documental y código que había dejado de tener una función dentro de la arquitectura final.

Como resultado de la revisión se identificaron inconsistencias en algunos contratos de la API, un problema funcional en el dashboard de Products, componentes heredados de la anterior arquitectura basada en JSON y determinadas oportunidades de optimización en el frontend. Estos casos fueron posteriormente corregidos y se documentan en las secciones siguientes.

CÓDIGO HEREDADO

Durante la auditoría se identificaron varios componentes pertenecientes a una etapa anterior del proyecto, cuando el frontend consumía archivos JSON generados a partir de las vistas analíticas de MySQL.

La migración hacia la API REST hizo que estos componentes dejaran de participar en el flujo de datos de la aplicación. Mantenerlos en el proyecto podía generar confusión sobre la arquitectura vigente y conservar dependencias que ya no tenían consumidores.

Entre los elementos identificados se encontraban los archivos JSON utilizados anteriormente para alimentar el frontend y el script `export_to_json.py`, encargado de extraer información de las vistas de MySQL y generar dichos archivos.

Estos componentes fueron eliminados una vez comprobado que ningún elemento activo del frontend o backend dependía de ellos. Las búsquedas realizadas sobre el proyecto confirmaron que las referencias restantes correspondían únicamente a documentación que todavía describía la arquitectura anterior.

También se identificaron funciones en `dashboard-controller.js` asociadas al antiguo procesamiento local de datasets. Entre ellas se encontraban `configureMonthlyDateFilter`, `getSelectedDatePeriods`, `getDatasetDateRange`, `getDefaultComparisonPeriods`, `filterRecordsByDate` y `normalizePeriodEnd`.

Estas funciones permitían calcular rangos, períodos de comparación y filtrados directamente sobre los registros disponibles en el frontend. Con la incorporación de la API, estas operaciones dejaron de ser necesarias dentro del cliente, ya que los períodos y parámetros de consulta son enviados al backend y el procesamiento de los datos se realiza sobre la fuente MySQL.

Antes de eliminarlas se verificó individualmente que no existieran consumidores activos fuera del conjunto de funciones heredadas. Una vez confirmado que no participaban en el flujo actual, fueron eliminadas.

La limpieza se realizó de forma incremental y cada eliminación fue seguida por pruebas locales de los cuatro dashboards. La aplicación continuó funcionando correctamente, confirmando que estos componentes ya no formaban parte de la arquitectura efectiva de la versión final.

INCONSISTENCIAS DETECTADAS

Durante la auditoría se identificaron algunas inconsistencias entre la implementación de determinados endpoints y los contratos definidos para la API. Estas diferencias no

impedían el funcionamiento general del sistema, pero reducían la consistencia de la interfaz y podían dificultar la validación de las respuestas.

La principal inconsistencia se encontraba en el endpoint de **Marketing**. A diferencia de los demás endpoints analíticos, la ruta no utilizaba inicialmente un `response_model` explícito. Esto hacía que la estructura de la respuesta no quedara formalmente asociada al schema correspondiente en la definición de la ruta.

La revisión de la respuesta real del endpoint mediante Swagger permitió comprobar además que Marketing retornaba información de tendencias que no estaba contemplada en el schema existente. La respuesta incluía las métricas principales del dashboard y una colección trends, cuyos elementos contenían período, canal, revenue, profit, marketing cost y ROAS.

Para corregir esta inconsistencia se actualizó el schema de Marketing incorporando una estructura específica para los elementos de tendencias y se añadió `MarketingResponseSchema` como `response_model` de la ruta.

La modificación permitió alinear tres elementos que debían representar el mismo contrato:

Respuesta generada



MarketingResponseSchema



Frontend

Posteriormente se ejecutó nuevamente el endpoint mediante Swagger y se comprobó que la respuesta continuaba siendo válida. También se verificó el funcionamiento del Marketing Dashboard desde el frontend local.

La auditoría permitió comprobar además que los contratos de los distintos dominios no debían forzarse a compartir exactamente la misma estructura interna. Cada dashboard posee necesidades específicas y, por ello, los schemas se mantienen separados por dominio, conservando únicamente la estructura general común de las respuestas de la API.

Estas correcciones no modificaron la lógica analítica del dominio, sino que regularizaron la definición y validación del contrato expuesto por la API.

CORRECCIÓN DE PRODUCTS

Durante las pruebas de integración se detectó un error en la carga del **Products Dashboard**. El resto de los dashboards podía obtener y representar correctamente la información, mientras que Products presentaba un fallo durante la construcción de sus gráficos.

La revisión mostró que el problema se encontraba en la forma en que el frontend procesaba los resultados financieros recibidos desde el endpoint de Products. La respuesta de la API contenía los resultados necesarios, pero la función encargada de generar los gráficos no estaba manejando correctamente determinados valores de la estructura recibida.

La corrección se realizó sobre la función de renderizado de los gráficos de Products, manteniendo la estructura de datos proporcionada por la API. Se reforzó el procesamiento de los resultados antes de construir los gráficos y se mantuvo el comportamiento dinámico según la dimensión seleccionada.

El dashboard contempla tres conjuntos de resultados:

- **Revenue**, ordenado por ingresos.
- **Profit**, ordenado por beneficio.
- **Units Sold** para la dimensión product, o **Margin** para las demás dimensiones.

Cada conjunto se limita a los ocho primeros resultados antes de ser enviado a la capa de visualización.

Después de aplicar la corrección, se realizaron pruebas tanto en el entorno local como mediante el flujo completo de la aplicación. Se verificó el funcionamiento del filtro temporal, el cambio de dimensión y la generación de los tres gráficos correspondientes.

La corrección permitió mantener la separación entre la obtención de datos y su representación visual: la API continúa siendo responsable de proporcionar los resultados analíticos, mientras que el frontend se limita a transformarlos en los elementos gráficos correspondientes.

OPTIMIZACIÓN

Una vez corregidos los problemas funcionales, se realizó una revisión orientada a mejorar el rendimiento percibido de la aplicación y reducir operaciones innecesarias durante la carga de los dashboards.

La revisión se concentró principalmente en el **Marketing Dashboard**, que presentaba los mayores tiempos de carga durante las pruebas de integración. Se analizaron las solicitudes realizadas por el frontend y el procesamiento ejecutado durante la inicialización de los filtros y la carga de los datos.

También se revisó el flujo de inicialización de los demás dashboards para mantener un comportamiento consistente y evitar operaciones heredadas de la anterior arquitectura basada en archivos locales.

Como parte de esta limpieza se eliminaron funciones y dependencias que ya no eran necesarias después de la migración hacia la API. Esto redujo la cantidad de procesamiento realizado directamente en el frontend y permitió que los filtros y consultas utilizaran directamente los parámetros enviados al backend.

Las optimizaciones se realizaron sin modificar los contratos de la API ni la estructura funcional de los dashboards. Después de cada modificación se realizaron pruebas locales para comprobar que Executive, Products, Customers y Marketing continuaran funcionando correctamente.

Como resultado, la aplicación presentó un flujo de carga más fluido durante las pruebas, especialmente en Marketing, manteniendo el mismo comportamiento funcional y la misma información presentada al usuario.

CAPITULO 4 - CONCLUSIÓN Y CIERRE

ESTADO FINAL DE LA ARQUITECTURA

Tras completar la implementación, integración y auditoría, Hartseer Analytics adopta una arquitectura en la que el frontend deja de acceder directamente a archivos de datos locales y consume la información analítica mediante la API REST.

El flujo final de datos queda establecido de la siguiente manera:

MySQL

↓

Repository

↓

Service

↓

Controller

↓

FastAPI REST API

↓

Frontend Data Service

↓

Dashboard Controller

↓

Dashboard Views

↓

Charts / UI

El **backend** se encuentra organizado en capas independientes, manteniendo separadas la exposición de endpoints, la coordinación de operaciones, la lógica de procesamiento, el acceso a datos y la infraestructura de conexión con MySQL.

El **frontend** utiliza la API como fuente de datos para los dashboards y envía los parámetros correspondientes a cada consulta, como períodos de análisis, dimensiones de agrupamiento y canales de marketing.

La comunicación entre ambos componentes se realiza mediante endpoints versionados bajo /api/v1. El backend se encuentra desplegado en Railway, mientras que el frontend se encuentra desplegado mediante GitHub Pages.

La arquitectura final elimina la dependencia de los archivos JSON utilizados durante las primeras etapas del proyecto y establece una separación clara entre la fuente de datos, la lógica de procesamiento y la capa de presentación.

Con esta estructura, los componentes principales de la aplicación quedan integrados en un único flujo de datos dinámico desde MySQL hasta la interfaz de los dashboards.

ESTADO DE LA API

La API de Hartseer Analytics se encuentra implementada y operativa como una capa REST encargada de proporcionar al frontend los datos necesarios para los dashboards de la aplicación.

La versión actual se encuentra expuesta bajo el prefijo `/api/v1` y contempla los siguientes endpoints:

Endpoint	Función
<code>/api/v1/health</code>	Comprobación del estado de la API
<code>/api/v1/executive</code>	Datos del Executive Dashboard
<code>/api/v1/products</code>	Datos del Products Dashboard
<code>/api/v1/customers</code>	Datos del Customers Dashboard
<code>/api/v1/marketing</code>	Datos del Marketing Dashboard

Los endpoints analíticos reciben los parámetros correspondientes a cada operación y obtienen la información directamente desde MySQL mediante las capas internas del backend.

Las respuestas se encuentran estructuradas mediante schemas específicos para cada dominio y mantienen una estructura general basada en success, data y meta. Esto permite que el frontend consuma los distintos recursos mediante contratos definidos y consistentes.

Durante la auditoría final se verificó el funcionamiento de los endpoints mediante Swagger y se validó posteriormente su consumo desde el frontend. También se corrigieron las inconsistencias detectadas en los contratos de respuesta y se comprobó nuevamente su funcionamiento después de las modificaciones.

La API constituye actualmente la fuente de datos utilizada por los cuatro dashboards de la aplicación, reemplazando completamente el mecanismo anterior basado en archivos JSON.

El backend se encuentra desplegado en **Railway**, mientras que el frontend utiliza los endpoints de producción para obtener los datos analíticos en tiempo de ejecución.

ESTADO DEL FRONTEND

El frontend se encuentra integrado con la API REST y utiliza sus endpoints como fuente de datos para los cuatro dashboards disponibles: **Executive, Products, Customers y Marketing**.

La aplicación mantiene una arquitectura SPA modular, en la que la obtención de datos se encuentra separada de la lógica de los dashboards y de la representación visual. Los filtros definidos en la interfaz generan las solicitudes correspondientes a la API, enviando los parámetros necesarios para cada operación.

La integración actual permite que:

- **Executive** consulte sus métricas según el período seleccionado.
- **Products** consulte los resultados según período y dimensión de agrupamiento.
- **Customers** consulte la información correspondiente al período seleccionado.
- **Marketing** consulte sus resultados según período y canal seleccionado.

Durante la auditoría se verificó el funcionamiento de los cuatro dashboards después de las modificaciones realizadas sobre el backend y el frontend. También se comprobó que la eliminación de las funciones asociadas al antiguo procesamiento local de datos no afectara el comportamiento de la aplicación.

El frontend continúa siendo responsable de la interacción con el usuario, la aplicación de filtros de interfaz, la coordinación de las vistas y la representación de los resultados mediante los componentes gráficos correspondientes. El procesamiento y la obtención de los datos analíticos se realizan mediante la API.

La versión desplegada se encuentra disponible mediante **GitHub Pages** y consume la API de producción desplegada en Railway, completando la integración entre la interfaz, el backend y la base de datos.

ELIMINACIÓN DE ARQUITECTURA TEMPORAL

Durante las primeras etapas del desarrollo, el frontend utilizó archivos JSON generados a partir de las vistas analíticas de MySQL como mecanismo temporal de suministro de datos. Este enfoque permitió validar inicialmente la lógica de los dashboards y su funcionamiento independiente del backend.

Con la implementación de la API REST y su conexión directa con MySQL, este mecanismo dejó de formar parte del flujo de ejecución de la aplicación. Por este motivo, durante la etapa de auditoría se eliminaron los componentes asociados a esta arquitectura temporal.

Entre los elementos retirados se encuentran:

- los archivos JSON utilizados como fuente de datos;
- `export_to_json.py`, utilizado para generar dichos archivos;
- las referencias funcionales asociadas al antiguo mecanismo de carga local;
- funciones del frontend destinadas a filtrar y procesar directamente los datasets locales.

La eliminación se realizó después de comprobar que estos componentes ya no tenían consumidores dentro de la aplicación. Las referencias restantes fueron revisadas para distinguir entre dependencias funcionales y menciones históricas dentro de la documentación.

Como resultado, la versión final ya no depende de archivos JSON ni de procesos de exportación para proporcionar información a los dashboards. El flujo de datos se realiza directamente desde MySQL, a través de la API REST, hasta el frontend.

Esta modificación completa la transición desde el mecanismo temporal utilizado durante el desarrollo hacia la arquitectura implementada para la versión 1.0.

ESTADO V1.0

Con la finalización de la integración entre el frontend, la API REST y MySQL, Hartseer Analytics alcanza el alcance definido para su **versión 1.0**.

La versión actual cuenta con:

- Base de datos MySQL integrada con el backend.
- API REST desarrollada con FastAPI.
- Endpoints versionados bajo /api/v1.
- Integración funcional de Executive, Products, Customers y Marketing.
- Contratos de respuesta definidos mediante schemas.
- Frontend SPA conectado a la API de producción.
- Corrección de los errores funcionales detectados durante la auditoría.
- Eliminación de código y componentes heredados de la arquitectura basada en JSON.
- Optimización del flujo de carga del frontend.
- Backend desplegado en Railway.
- Frontend desplegado en GitHub Pages.
- Documentación técnica actualizada para reflejar la arquitectura final.

La auditoría final permitió comprobar que los componentes activos del sistema corresponden a la arquitectura definida y que los elementos temporales utilizados durante las primeras etapas del desarrollo ya no forman parte de la ejecución de la aplicación.

De esta manera, **Hartseer Analytics v1.0 constituye una versión integrada y funcional de la plataforma**, en la que los dashboards consumen información analítica desde MySQL mediante una API REST, manteniendo separadas las capas de datos, procesamiento y presentación.

Esta versión establece la base técnica sobre la cual podrán incorporarse futuras funcionalidades sin alterar el funcionamiento de los componentes que conforman el alcance actual del proyecto.

